

IIT MADRAS — B.Sc PROGRAMMING & DATA SCIENCE

JAVA PROGRAMMING CONCEPTS

Week 1 Master Notes & Complete Reference Guide

Instructor: Prof. Madhavan Mukund (Chennai Mathematical Institute)

Coverage: Lectures 01 to 06 (Hardware Architecture, Types, Memory Management, Abstraction, OOP, Classes)

Format: Textbook-Grade Master Notes with Executable Java Cells & Dictionary

IITM Java Course — Week 1, Lecture 1: Introduction to Programming Concepts & Languages

Instructor: Prof. Madhavan Mukund (Chennai Mathematical Institute / IIT Madras) **Course:** Programming Concepts Using Java (B.Sc in Programming and Data Science)

Key Technical Terms Dictionary (Defined Upon First Appearance)

- **Programming Language:** A formal medium for communicating computational instructions step-by-step to a computer.
- **Main Memory (RAM):** Passive storage locations in hardware storing data and instructions; cannot perform arithmetic directly.
- **CPU Registers:** Ultra-fast storage slots inside the CPU core (R_1, R_2) where the ALU executes arithmetic and logical operations.
- **Arithmetic Logic Unit (ALU):** The CPU circuit responsible for executing arithmetic operations (addition, subtraction) and logical comparisons.
- **Compiler:** A translation program that converts high-level source code into low-level machine code in a single offline step before execution.
- **Interpreter:** A execution program that reads, translates, and executes high-level code line-by-line during runtime.
- **Imperative Programming:** A programming paradigm specifying **HOW** to compute via step-by-step state changes and loops.
- **Declarative Programming:** A paradigm specifying **WHAT** to compute via mathematical transformations (`filter`, `map`, `reduce`) without intermediate accumulators.
- **Java Streams API:** A declarative processing framework in Java for chaining sequence transformations cleanly.

1. What is a Programming Language?

A **language** is a fundamental medium of communication:

- **Natural Languages** (English, Hindi, Tamil, etc.): Used by humans to convey ideas, emotions, and abstract concepts to other humans.
- **Programming Languages:** Used by human programmers to convey **computational ideas and step-by-step instructions** to a machine (computer) that can execute them.

2. Low-Level Computer Architecture: Registers vs. Memory

To understand why high-level programming languages exist, we must first examine how computer hardware executes operations at a low level:

A. Main Memory (RAM)

- Main memory consists of a vast sequence of numbered storage locations (addresses).
- **Key Limitation:** Main memory is purely a passive storage location. You **cannot** perform arithmetic or logical operations directly inside RAM cells (e.g., you cannot directly take a value at memory location A and add it to memory location B inside RAM).

B. Central Processing Unit (CPU) & Arithmetic Logic Unit (ALU)

- All actual computation (addition, subtraction, logical comparisons) occurs inside the **ALU / CPU**.
- Inside the CPU, there is a very small set of ultra-fast storage slots called **Registers** ($R_1, R_2, R_3, \dots$).

C. The Low-Level Execution Cycle (Assembly/Machine Level)

To perform a basic addition like $x = y + z$, a traditional low-level machine must execute 4 distinct steps:

1. **FETCH 1:** Copy the value of variable y from its RAM location into CPU Register R_1 .
2. **FETCH 2:** Copy the value of variable z from its RAM location into CPU Register R_2 .
3. **COMPUTE:** Instruct the ALU to add the contents of R_1 and R_2 , storing the sum in R_1 .
4. **STORE:** Copy the contents of R_1 back to the RAM location allocated for variable x (to free up R_1 for future work and persist the result before power is lost).

Why Low-Level Machine Programming is Problematic:

- Writing programs in machine/assembly language requires dozens of manual register-management steps for a simple equation.
- It is extremely **tedious, error-prone**, and difficult to read or debug.
- It is easy to miss a register store step or overwrite a register accidentally.

Prof. Mukund's Words of Wisdom & Best Practices (Lecture 1)

1. The Hardware Evolution & Micro-Optimization Myth: "Modern computers and smartphones have vastly more resources than computers did 20 years ago. You do not need to worry about manual micro-optimizations in machine code. The small efficiency loss when moving to high-level languages is irrelevant compared to the gain in expressiveness."

2. Logical Errors vs. Micro-Efficiency: "Logical errors in code are far more damaging than minor execution speed differences. High-level code matches human intent, making logical errors much easier to detect and fix."

3. Avoid Magic Accumulator Variables: "When breaking down problems, avoid monolithic blocks of code with 'magic accumulator variables' (`mySum`). Instead, extract small, natural units of code (`isEven`, `square`) that explain why the code is correct without explaining low-level mechanics."

3. High-Level Abstraction & Computational Thinking

High-level programming languages bridge the gap between human thought and computer hardware by introducing **abstractions** that match computational thinking:

1. **Variables & Assignments:** Associates symbolic names with values (`x = 5`), shielding the programmer from raw RAM memory addresses.
2. **Conditional Execution:** Decision-making control flow (`if-else` branches).
3. **Repeated Execution:** Loops (`for`, `while`) to iterate over operations.
4. **Functions & Subroutines:** Grouping logical blocks of code into reusable procedures (e.g. `square(x)` or `factorial(n)`).
5. **Recursion:** Expressing inductive computations where a function calls itself with smaller inputs.
6. **Aggregate Data Structures:** Grouping multiple values into structured collections (Lists, Arrays, Dictionaries, Objects).

4. How High-Level Code Runs: Compilers vs. Interpreters

Since computer hardware only understands low-level machine code (0s and 1s), a bridge is needed to translate high-level code:

A. Compilers

- A **Compiler** is a program that reads the entire high-level source code and translates it into a low-level machine language program in **one single shot** before execution.
- **Examples:** C, C++, Java (`javac` compiles `.java` to JVM Bytecode `.class`).

B. Interpreters

- An **Interpreter** reads, translates, and executes the high-level source code **one line/instruction at a time**.
- **Example:** Python.

C. The Expressiveness vs. Efficiency Trade-off

- **Loss:** Moving to a high-level language relinquishes direct control over hardware registers, causing a minor loss in micro-efficiency.
- **Gain:** Immense gain in **expressiveness, code readability, maintainability**, and reduction in logical bugs.

5. Imperative vs. Declarative Programming Paradigms

Programming styles generally fall into two major paradigms:

| Feature | Imperative Style | Declarative / Functional Style | | :— | :— | :— | | **Core Focus** | **HOW** to compute (explicit step-by-step instructions). | **WHAT** to compute (relationship between input & output). | | **State Management** | Uses explicit temporary accumulator variables (`mySum`). | Avoids temporary accumulator state where possible. | | **Control Flow** | Manual `for/while` loops iterating element-by-element. | Mathematical pipelines (`filter`, `map`, `reduce`, `recursion`). | | **Verification** | Requires stepping through loop iterations to verify. | Transparent & self-describing based on problem definition. |

Detailed Code Example 1: Summing a List of Numbers

Imperative Approach in Java In the imperative approach, we explicitly initialize an accumulator variable (`mySum = 0`), iterate through every element using a loop, and add each element one-by-one.

```
import java.util.List;
import java.util.Arrays;

public class ImperativeSum {
    public static int sumList(List<Integer> list) {
        int mySum = 0; // Explicit intermediate accumulator variable
        for (int x : list) { // Explicit loop step-by-step iteration
            mySum += x;
        }
        return mySum;
    }

    public static void main(String[] args) {
        List<Integer> nums = Arrays.asList(10, 20, 30, 40);
        System.out.println("Imperative Sum: " + sumList(nums)); // Output: 100
    }
}

ImperativeSum.main(new String[]{});
```

Declarative Approach in Java (Streams API) In the declarative approach, we state **what** we want (reduce the sequence using addition) without managing manual loop counters or accumulator variables.

```
import java.util.List;
import java.util.Arrays;

public class DeclarativeSum {
    public static int sumList(List<Integer> list) {
        // Declarative sum using Stream reduction pipeline
        return list.stream().reduce(0, Integer::sum);
    }

    public static void main(String[] args) {
        List<Integer> nums = Arrays.asList(10, 20, 30, 40);
        System.out.println("Declarative Sum: " + sumList(nums)); // Output: 100
    }
}

DeclarativeSum.main(new String[]{});
```

Detailed Code Example 2: Sum of Squares of Even Numbers from 0 to N

Problem Description: Given an integer N , filter all even numbers in the range $[0, N]$, square each even number, and compute their total sum.

Imperative Approach

```
public class ImperativeEvenSquareSum {
    public static int sumEvenSquares(int n) {
        int mySum = 0;
        for (int x = 0; x <= n; x++) {
            if (x % 2 == 0) {
```



```

        mySum += (x * x);
    }
}
return mySum;
}

public static void main(String[] args) {
    int n = 10;
    // Even numbers: 0, 2, 4, 6, 8, 10
    // Squares: 0 + 4 + 16 + 36 + 64 + 100 = 220
    System.out.println("Imperative Sum (0.." + n + "): " + sumEvenSquares(n));
}
}

```

```
ImperativeEvenSquareSum.main(new String[]{});
```

Declarative Pipeline Approach (Filter → Map → Reduce) Here we break the problem into natural, reusable modular functions (isEven predicate, square transformation) and link them in a stream pipeline:

```

import java.util.stream.IntStream;

public class DeclarativeEvenSquareSum {
    // 1. Predicate helper function
    public static boolean isEven(int x) {
        return x % 2 == 0;
    }

    // 2. Transformation helper function
    public static int square(int x) {
        return x * x;
    }

    public static int sumEvenSquares(int n) {
        return IntStream.rangeClosed(0, n)
            .filter(DeclarativeEvenSquareSum::isEven) // Filter even numbers
            .map(DeclarativeEvenSquareSum::square)    // Square each element
            .sum();                                    // Sum total pipeline
    }

    public static void main(String[] args) {
        int n = 10;
        System.out.println("Declarative Pipeline Sum (0.." + n + "): " + sumEvenSquares(n));
    }
}

```

```
DeclarativeEvenSquareSum.main(new String[]{});
```

6. Comprehensive Summary & Key Takeaways

1. **Low-Level vs High-Level:** Computers operate on low-level memory and registers. High-level languages abstract hardware into variables, loops, and functions.
2. **Compilers vs Interpreters:** Compilers translate entire programs in one shot; interpreters translate line-by-line.
3. **Declarative Mindset:** Declarative programming encourages breaking complex operations into self-describing pipelines (filter → map → reduce), avoiding temporary accumulator variables.

IITM Java Course — Week 1, Lecture 2: Type Systems & Static Analysis

Instructor: Prof. Madhavan Mukund (Chennai Mathematical Institute / IIT Madras) **Course:** Programming Concepts Using Java (B.Sc in Programming and Data Science)

Key Technical Terms Dictionary (Defined Upon First Appearance)

- **Type System:** A structural framework in a programming language that classifies memory bit sequences into types and enforces valid operations.
- **Bit Sequence:** Raw binary strings of 0s and 1s stored in physical RAM cells.
- **Dimensional Analysis:** A scientific validation technique in physics (and static type checking) ensuring mathematical quantities cancel correctly ($v = a * t$).
- **Dynamic Typing:** A type system binding types to dynamic object values at runtime (e.g. Python), allowing type reassignment.
- **Static Typing:** A type system binding types permanently to variable declarations at compile-time (e.g. Java), preventing type mismatch bugs before execution.
- **Alan Turing's Halting Problem:** A landmark 1936 mathematical theorem proving that no general algorithm can determine if an arbitrary program will halt or loop infinitely.
- **Type Inference:** The static compiler feature (e.g. Java var) that automatically deduces variable types from initialization expressions.

1. Why Do We Need Types? (The Three Core Purposes)

At a physical hardware level, computer RAM stores only raw bit sequences (0s and 1s). Memory has no inherent understanding of whether a bit pattern represents an integer, a floating-point number, a character string, or a memory address.

Prof. Mukund highlights three fundamental reasons why programming languages impose **Type Systems**:

A. Interpreting Raw Binary Bit Strings

- Without a type, the bit string 01000001 could represent the integer 65, the ASCII character 'A', or a floating-point exponent.
- **Type Discipline:** Imposes a legal interpretation on raw memory bits, defining which operations (addition, string concatenation, logical AND) are valid.

B. Modeling Real-World Domain Concepts

- Types allow programmers to record their exact intent when building software.
- Rather than representing everything as raw primitives (floats or pairs), we create high-level domain types like Point2D, BankAccount, Customer, OR Transaction.

C. Early Error Detection (Dimensional Analysis Analogy)

- In physics, **Dimensional Analysis** validates equations before calculating numbers: "Velocity " $\text{left}\{m\}\{s\}\text{right}$ = "Acceleration " $\text{left}\{m\}\{s^2\}\text{right}$ "Time " $[s]$ If an equation wrote $\text{Velocity} = \text{Acceleration} * \text{Time}^2$, canceling dimensions produces meters (m), revealing a formula error.
- **Type Checking** acts as a dimensional sanity check in software, flagging incompatible operations (e.g., attempting to add an integer to a list) **before** execution.

Prof. Mukund's Words of Wisdom & Best Practices (Lecture 2)

1. The Building Construction Analogy & Early Error Cost: "In anything we do, the earlier we detect an error, the better it is—whether constructing a building or writing a program. Finding a flaw after the building is built requires tearing it down, which costs massive effort, time, and money. Static typing catches errors early at compile-time before your code ever runs."

2. Dimensionality Analogy from Physics: "Think of static type checking like dimensional analysis in physics. Checking if velocity equals acceleration times time ($v = a * t$) lets you cancel seconds to verify $\frac{m}{s}$. Type checking is your programming sanity check to ensure quantities going together make sense."

3. Domain-Specific Type Modeling: "In banking or geometric systems, do not treat everything as raw primitives (floats or pairs). Define high-level types like BankAccount OR Point2D to reflect ground truth and guarantee you cannot accidentally perform wrong operations on the wrong objects."

2. Dynamic Typing vs. Static Typing

Programming languages categorize type binding into **Dynamic** or **Static**:

| Feature | Dynamic Typing (e.g. Python) | Static Typing (e.g. Java, C++) | | :— | :— | :— | | **Type Binding Target** | Type is associated with the **value object**, not the variable name. | Type is associated with the **variable declaration** at compile time. | | **Type Mutability** | A variable `x` can hold 10 (int), then later 7.5 (float). | Variable `int x` is permanently bound to integer operations. | | **Uninitialized State** | Uninitialized variables have **no type** (referencing causes `NameError`). | Variables must be declared with an explicit type (`int x;`). | | **Error Catching** | Caught at **runtime** (user-facing crashes during execution). | Caught at **compile-time** (fast static analysis before running). |

The Factor List Typo Bug Demonstration

Prof. Mukund demonstrates a classic pitfall in dynamically typed languages:

Python Typo Bug Example

```
def factors(n):
    factor_list = [] # Initialized empty list
    for i in range(1, n + 1):
        if n % i == 0:
            factor_lst.append(i) # TYPO: Spelled 'factor_lst' instead of 'factor_list'!
    return factor_list
```

- **Why Python Fails to Catch This:** Because Python uses dynamic typing without variable declarations, it assumes `factor_lst` is a brand new variable. Python does not raise a syntax error during parsing, causing `factor_list` to stay empty and producing silent runtime logical bugs!

Java Statically Caught Equivalent In Java, every variable must be declared. A misspelling halts the compiler immediately:

```
import java.util.ArrayList;
import java.util.List;

public class StaticTypeCheckFactors {
    public static List<Integer> getFactors(int n) {
        List<Integer> factorList = new ArrayList<>();
        for (int i = 1; i <= n; i++) {
            if (n % i == 0) {
                factorList.add(i); // If typed as factorLst, javac throws a compilation error instantly!
            }
        }
        return factorList;
    }

    public static void main(String[] args) {
        System.out.println("Factors of 28: " + getFactors(28));
    }
}

StaticTypeCheckFactors.main(new String[]{});
```

3. Alan Turing's Halting Problem & Limits of Program Checking

A common question arises: **Why can't we write an automated program that checks whether another program is 100% correct?**

Alan Turing's Undecidability Theorem (1936):

It is mathematically impossible to construct a general algorithm that inspects an arbitrary program P and input X and decides whether $P(X)$ will eventually halt (terminate) or loop forever.

- **Practical Consequence:** Because full automated program verification is mathematically impossible, static type systems provide an indispensable, decidable framework to eliminate a massive category of errors (type mismatches, illegal operations) automatically.

4. Static Typing without Explicit Declarations: Type Inference

- Static typing does **not** strictly require verbose manual declarations if the compiler supports **Type Inference**.

- **Type Inference:** The compiler deduces types statically from context: $x = 7 \Rightarrow$ **compiler infers x is int**. $y = x + 15 \Rightarrow$ compiler infers y is int.
- Modern Java (Java 10+) supports type inference via the `var` keyword for local variables!

```
public class TypeInferenceDemo {
    public static void main(String[] args) {
        // Java 'var' keyword infers static type at compile time
        var message = "Type Inference in Java 17"; // Inferred as String
        var count = 42;                          // Inferred as int
        var pi = 3.14159;                        // Inferred as double

        System.out.println(message + " | Count: " + count + " | Pi: " + pi);
    }
}
```

```
TypeInferenceDemo.main(new String[]{});
```

5. Summary & Key Takeaways

1. **Type Systems:** Convert raw bits into structured domain entities and validate operation rules.
2. **Building Construction Analogy:** Catching errors at compile time saves massive cost compared to discovering runtime bugs after deployment.
3. **Type Inference:** Java supports `var` local variable type inference while maintaining 100% static type safety.

IITM Java Course — Week 1, Lecture 3: Memory Management (Stack, Heap & GC)

Instructor: Prof. Madhavan Mukund (Chennai Mathematical Institute / IIT Madras) **Course:** Programming Concepts Using Java (B.Sc in Programming and Data Science)

Key Technical Terms Dictionary (Defined Upon First Appearance)

- **Stack Memory:** A contiguous, LIFO (Last-In, First-Out) memory region dedicated to tracking method execution frames and primitive local variables.
- **Heap Memory:** A dynamic RAM pool used for allocating objects and arrays that outlive method execution frames.
- **Activation Record (Stack Frame):** A structured memory block pushed onto the call stack for every method invocation, containing parameters, local variables, a Control Link, and a Return Value Link.
- **Control Link (Dynamic Link):** A memory pointer inside an activation record pointing to the caller's previous stack frame.
- **Return Value Link:** A memory location slot in the caller's stack frame where a returning function stores its result.
- **malloc():** A manual C library function (memory allocation) that requests dynamic memory blocks from the Heap.
- **free():** A manual C library function that deallocates heap memory and returns it to the free memory pool.
- **Memory Leak:** A critical defect where allocated heap memory is abandoned without being deallocated, draining available RAM over time.
- **Garbage Collection (GC):** An automatic runtime process in Java/Python that scans the Heap and reclaims memory occupied by unreachable objects.
- **Mark-and-Sweep:** A 2-phase GC algorithm that marks all reachable objects starting from root references, then sweeps (deallocates) unmarked objects.

1. Defining Stack Memory vs. Heap Memory from First Principles

During program execution, memory is divided into two distinct regions to handle different variable lifecycles:

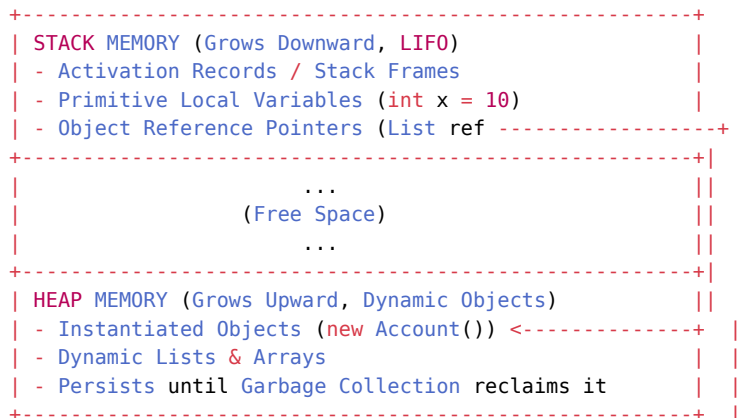
A. What is Stack Memory?

- **Definition:** A continuous block of memory managed in a **Last-In, First-Out (LIFO)** manner, specifically dedicated to tracking function calls and local variables.

- **What it Stores:** Local primitive variables (`int`, `double`), method parameters, and object reference pointers (`myList`).
- **Allocation & Deallocation:** Automatically managed by the execution call stack. When a method is called, its frame is pushed onto the stack; when the method returns, its frame is popped and memory is immediately reclaimed.

B. What is Heap Memory?

- **Definition:** A large pool of dynamic memory used for objects and arrays whose size or lifecycle cannot be predicted at compile time.
- **What it Stores:** Instantiated objects (`new Developer()`) and arrays (`new int[100]`).
- **Allocation & Deallocation:** Dynamically allocated using `new`. Heap memory persists **beyond** method execution exits. Deallocated via manual `free` (in C) or automatic Garbage Collection (in Java/Python).



Important Distinction: Operating system **Heap Memory** is a region of dynamic RAM. It has **no relation** to the Heap Data Structure (Binary Heap) used for Priority Queues!

Prof. Mukund's Words of Wisdom & Best Practices (Lecture 3)

1. The Water Tank Analogy for Memory Leaks: "Think of your free memory space as a water tank. Every time you ask for dynamic storage, you open a tap and draw water. A memory leak means water is emptying out without being replenished back to the tank. Over time, large systems like web browsers or operating systems slow down or crash because they leak memory resources."

2. Reference Passing Warning: In-Place Mutation vs Reassignment: "When passing objects or arrays to functions by reference, remember: you can mutate the object in-place (`arr[0] = x`) to produce intentional side-effects outside. However, if you reassign the parameter inside (`arr = new_arr`), you only change what the local parameter variable points to—it will NOT update the caller's reference variable outside!"

3. Garbage Collection Pause Trade-off: "Automatic Garbage Collection removes the headache of manual memory management. But remember the tradeoff: GC runs in the background and may cause unexpected slowdowns outside your control because execution must pause during sweep cycles."

2. Scope vs. Lifetime of Variables

- **Scope:** The static region of program text where a variable name can be legally accessed.
- **Lifetime:** The dynamic duration during program execution that storage remains allocated for a variable in memory.

Hole in Scope: A variable's lifetime can exceed its scope. When function f calls function g , local variables of f remain allocated on f 's stack frame, but enter a temporary hole in scope until g finishes and pops.

3. Activation Records (Stack Frames)

Each function invocation pushes an **Activation Record** containing:

1. **Local Variables & Parameters:** Memory for parameter values and local block variables.
2. **Control Link (Dynamic Link):** Pointer to the caller's activation record at the bottom of the stack.

3. Return Value Link: Pointer indicating where to store the returned result in the calling function's frame.

```
public class CallStackDemo {
    public static int factorial(int n) {
        if (n <= 0) {
            return 1; // Base case: pops frame from stack and returns result
        }
        return n * factorial(n - 1); // Pushes new frame on stack
    }

    public static void main(String[] args) {
        int result = factorial(4);
        System.out.println("Factorial(4) = " + result);
    }
}
```

```
CallStackDemo.main(new String[]{});
```

4. Parameter Passing: Call-by-Value vs. Call-by-Reference

Call-by-Value (Primitives)

- A standalone copy of the primitive value is passed to the parameter slot. Changes inside the function do not affect the caller.

Call-by-Reference / Reference-Passing (Objects & Arrays)

- The reference (pointer address) is passed.
- **In-Place Mutation:** Mutating internal properties (`arr[0] = 777`) alters the shared heap object (side-effect).
- **Reference Reassignment:** Reassigning `arr = new int[]` changes the local parameter pointer only, leaving the caller's reference unaffected.

```
import java.util.Arrays;

public class ParameterPassingDemo {
    public static void modifyPrimitive(int x) {
        x = 999; // Modifies local copy only
    }

    public static void mutateArray(int[] arr) {
        arr[0] = 777; // Mutates shared heap memory
    }

    public static void reassignArray(int[] arr) {
        arr = new int[]{100, 200, 300}; // Points to fresh heap allocation
    }

    public static void main(String[] args) {
        int num = 10;
        modifyPrimitive(num);
        System.out.println("Primitive after modify: " + num); // 10

        int[] myArr = {1, 2, 3};
        mutateArray(myArr);
        System.out.println("Array after mutate: " + Arrays.toString(myArr)); // [777, 2, 3]

        reassignArray(myArr);
        System.out.println("Array after reassign: " + Arrays.toString(myArr)); // [777, 2, 3]
    }
}
```

```
ParameterPassingDemo.main(new String[]{});
```

5. Memory Reclamation: Manual vs. Automatic Garbage Collection

Manual Memory Management (C/C++)

- Programmer calls `malloc()` / `free()`. Failure to free causes **Memory Leaks** (water tank running empty).

Automatic Garbage Collection (Java & Python)

- **Mark-and-Sweep Algorithm:**

1. **Mark:** Traverses references starting from active Stack frames (Root Set). Marks every reachable object on the Heap.
2. **Sweep:** Scans the Heap and reclaims memory for all **unmarked** (unreachable) objects back to the free memory pool.

6. Comprehensive Summary & Key Takeaways

1. **Stack Memory:** Stores activation records, local primitives, and reference pointers in a LIFO manner.
2. **Heap Memory:** Stores dynamic objects and arrays; persists beyond function execution.
3. **Garbage Collection:** Mark-and-Sweep reclaims unreachable heap objects automatically.

IITM Java Course — Week 1, Lecture 4: Abstraction, Modularity & Program Refinement

Instructor: Prof. Madhavan Mukund (Chennai Mathematical Institute / IIT Madras) **Course:** Programming Concepts Using Java (B.Sc in Programming and Data Science)

Key Technical Terms Dictionary (Defined Upon First Appearance)

- **Abstraction:** The design principle of hiding low-level implementation details behind clean, high-level public interfaces.
- **Modularity:** Decomposing a complex system into independent, self-contained units (modules) that interact through public contracts.
- **Stepwise Program Refinement:** Top-down problem decomposition where a broad task is recursively broken into smaller sub-tasks.
- **Data Refinement:** The process of upgrading internal data structures (e.g. adding transaction logs) while insulating external callers.
- **Component Interface:** The static, syntactic contract defining method signatures (names, parameter types, return types).
- **Component Specification:** The conceptual/behavioral contract defining **what** a component guarantees mathematically.
- **Abstract Data Type (ADT):** A mathematical model for data types defined by behavior (operations) rather than concrete memory layout.
- **Priority Queue ADT:** An ADT supporting `insert(item, priority)` and `removeMax()` operations.

1. What is Abstraction and Modularity?

Writing large software systems is complex. The only reliable way to manage software complexity is through **Abstraction** (hiding unnecessary implementation details) and **Modularity** (building software in self-contained, decoupled parts).

Program Refinement (Top-Down Problem Decomposition)

When facing a complex problem, we start with a high-level description of what needs to be done and refine it step-by-step into concrete sub-tasks:

graph TD;

```
A["Task: Print First 1000 Prime Numbers"] --> B["1. Create Prime Storage Table P"];
A --> C["2. Calculate & Fill First 1000 Primes into P"];
A --> D["3. Print Contents of Table P"];
B --> B1["Declare Integer Array P[1000]"];
C --> C1["Loop k = 0..999: Calculate k-th prime and store in P[k]"];
D --> D1["Loop k = 0..999: Print P[k]"];
```


Prof. Mukund's Words of Wisdom & Best Practices (Lecture 4)

1. The Alumni Event Organizing Analogy: "If someone tells you 'Organize a college alumni event next week', at that high level it is overwhelming. Refinement means decomposing it into concrete sub-tasks: booking an auditorium, estimating attendance, ordering chairs, setting up lighting, sound, and catering. Once decomposed, sub-tasks can be delegated to separate teams."

2. The Role of Prototypes: "Always build a simple prototype implementation for each component first to verify that components interact properly and fulfill system requirements. The prototype does not need to be fast or scalable initially—it proves your architecture is sound before you optimize."

3. Interface Decoupling & Component Upgrades: "When you upgrade an internal component (e.g. replacing a simple sorting routine with an efficient one), as long as you preserve the public interface signature, no other component interacting with it needs to change."

2. Data Refinement & Cascading Dependencies

- **Data Refinement:** As system requirements evolve, data representations change.
- **Example: Banking System Requirement Shift: Initial Model:** Store account balance as a single float balance. deposit() and withdraw() mutate balance. printStatement() prints balance. **Refined Requirement:** Printing a statement requires displaying the full transaction history log. **Cascading Impact:** Changing the internal data representation (adding a List<Transaction>) forces updates across deposit(), withdraw(), and transfer() functions.

Encapsulation Goal: Modularity isolates components so data refinement changes stay encapsulated inside module boundaries without breaking external caller code.

3. Interface vs. Behavioral Specification

A. Component Interface (Static Structural Contract)

- Defines **how** other components interact with a module.
- In Java, an interface specifies method names, parameter types, order, and return types (**Function Signature**).

B. Component Specification (Behavioral Contract)

- Defines **what** the component does conceptually/mathematically.
- E.g., withdraw(account, amount) must decrement amount from account balance if balance >= amount.

Detailed Code Example: Abstract Data Type (Priority Queue Interface vs Implementation)

```
// Public Interface Contract
interface PriorityQueueADT<T> {
    void insert(T item, int priority);
    T removeMax();
    boolean isEmpty();
}

import java.util.PriorityQueue;
import java.util.Comparator;

class Task {
    String name;
    int priority;

    public Task(String name, int priority) {
        this.name = name;
        this.priority = priority;
    }

    @Override
    public String toString() {
        return name + " (Priority: " + priority + ")";
    }
}
```



```

}

public class PriorityQueueDemo implements PriorityQueueADT<Task> {
    private PriorityQueue<Task> pq = new PriorityQueue<>((Comparator.comparingInt((Task t) ->
t.priority).reversed()));

    @Override
    public void insert(Task item, int priority) {
        pq.add(item);
    }

    @Override
    public Task removeMax() {
        return pq.poll();
    }

    @Override
    public boolean isEmpty() {
        return pq.isEmpty();
    }

    public static void main(String[] args) {
        PriorityQueueDemo queue = new PriorityQueueDemo();
        queue.insert(new Task("Regular Sync", 1), 1);
        queue.insert(new Task("Security Patch", 10), 10);
        queue.insert(new Task("UI Bugfix", 5), 5);

        System.out.println("Highest Priority Task Served: " + queue.removeMax());
        System.out.println("Next Priority Task Served: " + queue.removeMax());
    }
}

PriorityQueueDemo.main(new String[]{});

```

4. Comprehensive Summary & Key Takeaways

1. **Program & Data Refinement:** Decomposes tasks into modular components while isolating data representation shifts.
2. **Interfaces & Specifications:** Interfaces enforce structural contracts; specifications define behavioral expectations.
3. **Prototypes:** Verify component interaction early before optimizing internal algorithms.

IITM Java Course — Week 1, Lecture 5: Foundations of Object-Oriented Programming (OOP)

Instructor: Prof. Madhavan Mukund (Chennai Mathematical Institute / IIT Madras) **Course:** Programming Concepts Using Java (B.Sc in Programming and Data Science)

Key Technical Terms Dictionary (Defined Upon First Appearance)

- **Object-Oriented Programming (OOP):** A programming paradigm organized around data entities (objects) that bundle data attributes with operations operating on that data.
- **Simula 67:** Developed in the 1960s by Ole-Johan Dahl and Kristen Nygaard, it was the first programming language to introduce classes, objects, and dynamic lookup for discrete-event simulations.
- **Dynamic Lookup (Polymorphism):** The runtime mechanism where calling a method $f()$ on a parent type reference dynamically dispatches the specific subtype's implementation stored in memory.

- **Subtyping:** A static type hierarchy relationship where subtype A is a specialized version of parent type B ($A \subseteq B$). Wherever B is expected, A can be legally passed.
- **Inheritance:** The structural reuse of implementation code (fields and methods) defined in parent superclasses.
- **Heterogeneous Queue:** A data structure collection capable of storing elements of different concrete subtypes under a unified parent superclass type.

1. Historical Origins: Simula 67 & The Heterogeneous Event Queue

- **Simula (1960s):** First programming language to introduce **Objects** and **Classes** for discrete-event simulations.
- **The Simulation Problem:** In a real-time system simulation, diverse event types occur (e.g., `CustomerArrivalEvent`, `ServerFailureEvent`, `PacketDepartureEvent`).
- **The Dual Challenge:**
 1. How to store diverse event types inside a single well-typed event queue?
 2. How to execute `event.simulate()` without writing cumbersome `if-else` type-checking branches?
- **The Solution:** Every event is a **subtype** of a general `SimulationEvent` superclass, and each event object knows how to simulate itself (**Dynamic Lookup**).

Prof. Mukund's Words of Wisdom & Best Practices (Lecture 5)

1. The Mindset Shift: Data-Centric Programming: "Traditional programming focuses on control flow—writing sequences of statements and passing data around between functions. Object-oriented programming reverses the priority: ask first 'What data entities do we need to manage?' and attach functions to the data on which they operate."

2. The Subset Analogy for Subtyping ($A \subseteq B$): "Think of subtyping like mathematical subsets. If A is a subtype of B , then every A is also a B ($A \subseteq B$). Subtype A has as many capabilities as B and more. Wherever B is legally expected, A can be substituted."

3. The Crucial Distinction: Subtyping vs Inheritance (Deque Counterexample): "Inheritance is a relationship of implementations (reusing written code). Subtyping is a static relationship of interfaces (guaranteeing legal function calls). A `Stack` reuses `Deque`'s implementation, but is NOT a subtype of `Deque` because a `Stack` cannot legally allow `deleteRear()`!"

2. The Four Pillar Aspects of Object-Oriented Programming

graph TD;

```

    OOP["Object-Oriented Programming"] --> A["1. Abstraction"];
    OOP --> B["2. Subtyping (Hierarchy)"];
    OOP --> C["3. Dynamic Lookup (Polymorphism)"];
    OOP --> D["4. Inheritance (Code Reuse)"];

```

1. **Abstraction:** Separating public interface operations from private internal data implementations.
2. **Subtyping:** Type hierarchy where subtype A is a specialized version of parent type B ($A \subseteq B$). Wherever B is expected, A can be legally passed.
3. **Dynamic Lookup (Polymorphism):** At runtime, calling a method $f()$ on a reference variable of parent type B dynamically dispatches the specific subtype A 's implementation based on the actual object stored in memory.
4. **Inheritance:** Reusing implementation details (fields & methods) defined in parent superclasses.

Code Example 1: Simula Heterogeneous Event Queue Simulator

```

import java.util.LinkedList;
import java.util.Queue;

// Base Event Class
abstract class SimulationEvent {
    protected double timestamp;

    public SimulationEvent(double timestamp) {
        this.timestamp = timestamp;
    }

    // Polymorphic method contract

```

```

    public abstract void simulate();
}

// Subtype 1: Customer Arrival
class CustomerArrivalEvent extends SimulationEvent {
    private String customerName;

    public CustomerArrivalEvent(double timestamp, String customerName) {
        super(timestamp);
        this.customerName = customerName;
    }

    @Override
    public void simulate() {
        System.out.println "[" + timestamp + "s] Customer Arrived: " + customerName);
    }
}

// Subtype 2: Server Maintenance
class ServerMaintenanceEvent extends SimulationEvent {
    private int serverId;

    public ServerMaintenanceEvent(double timestamp, int serverId) {
        super(timestamp);
        this.serverId = serverId;
    }

    @Override
    public void simulate() {
        System.out.println "[" + timestamp + "s] Server #" + serverId + " undergoing maintenance.");
    }
}

public class SimulaEventQueueDemo {
    public static void main(String[] args) {
        // Heterogeneous Queue holding base type SimulationEvent references
        Queue<SimulationEvent> eventQueue = new LinkedList<>();
        eventQueue.add(new CustomerArrivalEvent(1.0, "Devendra"));
        eventQueue.add(new ServerMaintenanceEvent(2.5, 101));
        eventQueue.add(new CustomerArrivalEvent(3.2, "Anita"));

        // Simulation Loop: Dynamic Lookup dispatches proper simulate() method at runtime!
        while (!eventQueue.isEmpty()) {
            SimulationEvent e = eventQueue.poll();
            e.simulate();
        }
    }
}

SimulaEventQueueDemo.main(new String[]{});

```

Code Example 2: Employee vs. Manager (Inheritance & Subtyping from Slides)

As shown in Prof. Mukund's slides, a Manager retains basic personal data and joining date from Employee, while extending it with dateOfPromotion and seniority.

```

class Employee {
    protected String name;
    protected String dateOfJoining;

    public Employee(String name, String dateOfJoining) {
        this.name = name;
        this.dateOfJoining = dateOfJoining;
    }

    public void getDetails() {

```

```

        System.out.println("Employee: " + name + " | Joined: " + dateOfJoining);
    }
}

// Manager inherits from Employee (Code Reuse + Subtyping)
class Manager extends Employee {
    private String dateOfPromotion;
    private int seniority;

    public Manager(String name, String dateOfJoining, String dateOfPromotion, int seniority) {
        super(name, dateOfJoining);
        this.dateOfPromotion = dateOfPromotion;
        this.seniority = seniority;
    }

    @Override
    public void getDetails() {
        System.out.println("Manager: " + name + " | Joined: " + dateOfJoining +
            " | Promoted: " + dateOfPromotion + " | Seniority Level: " + seniority);
    }
}

public class EmployeeManagerDemo {
    public static void main(String[] args) {
        Employee emp = new Employee("Rahul", "2021-06-15");
        Employee mgr = new Manager("Priya", "2018-03-10", "2023-01-01", 5);

        emp.getDetails();
        mgr.getDetails(); // Dynamic Lookup dispatches Manager's getDetails()
    }
}

EmployeeManagerDemo.main(new String[]{});

```

3. Subtyping vs. Inheritance: The Deque Counterexample

Crucial Distinction (Prof. Mukund):

Subtyping: A relationship between **Interfaces** (is *A* usable wherever *B* is expected?).

Inheritance: A relationship between **Implementations** (does *A* reuse code written in *B*?).

The Deque Counterexample

- A **Double-Ended Queue (Deque)** supports 4 operations: `insertFront`, `deleteFront`, `insertRear`, `deleteRear`.
- A **Stack** can be easily implemented by delegating to a Deque's `insertFront` and `deleteFront`.
- A **Queue** can be implemented by delegating to a Deque's `insertRear` and `deleteFront`.
- **Is Stack a Subtype of Deque? NO! A Deque supports `deleteRear()`. If Stack were a subtype of Deque, calling `stack.deleteRear()` would be legal, breaking Stack semantics.** Therefore, Stack **inherits implementation** from Deque, but is **NOT a subtype** of Deque!

4. Comprehensive Summary & Key Takeaways

1. **Origins:** Simula 67 solved heterogeneous simulation queues using dynamic method dispatch.
2. **4 Pillars of OOP:** Abstraction, Subtyping, Dynamic Lookup, and Inheritance.
3. **Employee-Manager:** Manager extends Employee by reusing fields while augmenting promotion details.
4. **Subtyping vs Inheritance:** Subtyping governs interface compatibility; Inheritance governs code reuse.

IITM Java Course — Week 1, Lecture 6: Classes, Objects & Static Privacy Enforcement

Instructor: Prof. Madhavan Mukund (Chennai Mathematical Institute / IIT Madras) **Course:** Programming Concepts Using Java (B.Sc in Programming and Data Science)

Key Technical Terms Dictionary (Defined Upon First Appearance)

- **Class (Template):** A static blueprint defining the instance variables (data structure) and methods (behavioral operations) for objects.
- **Object (Instance):** A concrete, dynamically allocated entity in heap memory built from a class template.
- **Constructor:** A special class initialization method executed implicitly during `new` allocation to populate initial state.
- **Encapsulation:** Restricting direct access to internal data fields while exposing public methods to mutate or read state.
- **Access Modifiers:** Keywords (`private`, `package-private`, `protected`, `public`) enforcing compile-time access boundaries.
- **Subtyping Hierarchy:** A subtype specialization relationship where child classes extend parent behavior while preserving interface compatibility.

1. Class (Template) vs. Object (Instance)

- **Class:** A static template/blueprint defining instance variables (data layout) and public methods (behavioral operations).
- **Object:** A dynamic concrete instance allocated in heap memory with its own state (`instance.variable`).
- **Constructor:** A special initialization routine executed implicitly when an object is instantiated (`new ClassName(...)`).

Prof. Mukund's Words of Wisdom & Best Practices (Lecture 6)

1. Method Invocation as Sending Messages: "In object-oriented programming, do not think of passing data to an external function. Instead, send a message to an object and tell it to operate on itself (`list.sort()` vs `sorted(list)`). Every object receives the message and executes the action on its internal state."

2. The Failure of Relying on Programmer Discipline: "In dynamic languages like Python, instance variables cannot be made truly private (`p.x = 4` direct mutation breaks abstraction). You could write a manual advising your team never to touch instance variables, but relying on good sense fails over long software lifespans across multiple developer groups. Static access control (`private`, `protected`, `public`) in Java enforces privacy at compile-time instead of relying on programmer discipline."

3. Data Refinement & Method Co-location: "When you change internal data representation (e.g., Cartesian (x, y) to Polar (r, θ)), because all functions manipulating that data sit inside the class template alongside the data, you can update them in one place transparently without breaking any external callers."

2. Refactoring Internal Representation: Cartesian vs. Polar Point

Prof. Mukund presents a classic encapsulation example: changing a `Point` class internal storage from Cartesian (x, y) to Polar coordinates (r, θ) .

Mathematical Formulas for Conversion:

"Cartesian " $(x, y) \implies r = \sqrt{x^2 + y^2}$, quad $\theta = \arctan(\{y\}\{x\})$ "Polar " $(r, \theta) \implies x = r \cos(\theta)$, quad $y = r \sin(\theta)$

- **Public Interface Contract:** `translate(dx, dy)` `distanceFromOrigin()`

Encapsulation Goal: The caller should be able to invoke `translate()` and `distanceFromOrigin()` without knowing whether internal storage is (x, y) or (r, θ) .

Detailed Code Example: Java Point Class with Encapsulated Storage

```

public class Point {
    // Private instance variables (hidden from external direct mutation)
    private double x;
    private double y;

    // Constructor
    public Point(double x, double y) {
        this.x = x;
        this.y = y;
    }

    // Public Method: Translate point by dx, dy
    public void translate(double dx, double dy) {
        this.x += dx;
        this.y += dy;
    }

    // Public Method: Distance from origin using Pythagoras theorem
    public double distanceFromOrigin() {
        return Math.sqrt(x * x + y * y);
    }

    public double getX() { return x; }
    public double getY() { return y; }

    public static void main(String[] args) {
        Point p = new Point(3.0, 4.0);
        System.out.println("Initial distance from origin: " + p.distanceFromOrigin()); // 5.0

        p.translate(2.0, 4.0); // New coords: (5.0, 8.0)
        System.out.println("Translated Coords: (" + p.getX() + ", " + p.getY() + ")");
        System.out.println("New distance from origin: " + String.format("%.2f", p.distanceFromOrigin()));
    }
}

Point.main(new String[]{});

```

3. Privacy Enforcement: Python Flaws vs. Java Access Specifiers

Flaws in Dynamic Languages (e.g. Python)

- In Python, instance variables cannot be made truly private (`p.x = 4` can be mutated directly from caller code).
- If the maintainer renames internal fields (e.g., from `width/height` to `wd/ht`), external callers break at runtime with `AttributeError`.

Static Privacy Specifiers in Java

Access Modifier	Visible Inside Class?	Visible in Subclasses?	Visible in Package?	Visible Everywhere?
<code>private</code>	Yes	No	No	No
package-private (default)	Yes	No	Yes	No
<code>protected</code>	Yes	Yes	Yes	No
<code>public</code>	Yes	Yes	Yes	Yes

4. Subtyping Hierarchy Example: Rectangle & Square

A Square is a specialized Rectangle where `width == height`.

```

class Rectangle {
    protected double width;
    protected double height;

    public Rectangle(double width, double height) {
        this.width = width;
        this.height = height;
    }

    public double area() {
        return width * height;
    }
}

```

```
    public double perimeter() {
        return 2 * (width + height);
    }
}

// Square inherits from Rectangle
class Square extends Rectangle {
    public Square(double side) {
        super(side, side); // Invoke parent Rectangle constructor
    }
}

public class SubtypeDemo {
    public static void main(String[] args) {
        Rectangle rect = new Rectangle(5.0, 10.0);
        System.out.println("Rectangle Area: " + rect.area() + " | Perimeter: " + rect.perimeter());

        Rectangle sq = new Square(4.0); // Subtype Polymorphism: Square stored in Rectangle reference
        System.out.println("Square Area: " + sq.area() + " | Perimeter: " + sq.perimeter());
    }
}

SubtypeDemo.main(new String[]{});
```

5. Comprehensive Summary & Key Takeaways

1. **Classes vs Objects:** Classes are static templates; objects are concrete heap-allocated instances.
 2. **Encapsulation:** Private fields prevent external code from depending on internal representations.
 3. **Static Access Control:** Java's private, protected, and public keywords enforce compile-time abstraction boundaries.
-